Tools to implement, monitor and enforce what we covered

Enterprise Agentic Systems: Foundations, CSOB, Prague. A companion to the Day 4 "where this lives" page.

The four days gave you a set of practices: limit what an agent can reach, keep decisions outside the agent, trace what it does, cap what it costs, test it again after every change, and make sure a named person owns each control. This guide shows the tools that help with each one. It starts with the four products you found, then adds the other main options in each category.

Read this first. Tools are examples of a category, not recommendations. A tool is not a control: the control is the tool plus a named owner plus evidence that it still works. Product details come from vendor pages and recent comparison articles, which are marketing in places. Owners and licences change quickly in this market, so confirm anything you rely on with CSOB security, compliance and procurement.

Four jobs, four kinds of tool

Job	What it answers	Kinds of tool	Days
Implement and test	Does it behave as specified, and does it still after a change?	Evaluation frameworks, CI quality gates, prompt versioning, pinned models	2, 3
Observe	What did the agent do, what did it cost, and who gets told at 02:00?	Tracing, cost and latency monitoring, alerts, human review queues	3, 4
Enforce	What stops it doing the wrong thing while it runs?	Guardrail services, LLM gateways, MCP gateways, spend caps, access control	2, 3, 4
Prove and govern	Can we show the controls exist, are owned, and still work?	Red teaming, risk and policy platforms, inventory of AI use, audit trails	4

The wall sentence from Day 4 applies to every row: **a control nobody owns is a wish.**

The four tools you found

All four are observability and evaluation products first. None of the pages describes a full runtime enforcement layer, so you would pair any of them with the guardrail and gateway tools later in this guide.

Tool	What the page says it does	Notable points	Ask the vendor
Datadog Agent Observability	Traces every decision, tool call and handoff in single and multi-agent workflows, linked to backend services, APIs and infrastructure. Tracks tokens, latency and cost with real-time alerts. Built-in and custom evaluations including hallucination checks. Experiments compare prompts and models on the same datasets before rollout.	Natural fit if the organisation already runs Datadog. Claims 1,000+ integrations, including OpenAI, Anthropic, Bedrock, LangChain and CrewAI. Mentions security against prompt injection and data leaks.	The page lists a separate "AI Guard" product but gives no detail. What does it block, and where does it run?
Orq.ai observability and monitoring	Traces executions across agents, models, tools and workflows, with OpenTelemetry and SDK integration. Real-time usage, cost and reliability data. Alerts and routing of problem executions to human review, with an audit trail.	Configurable logging and PII masking, and permission policies on who sees traces. Links to separate AI Evaluation and AI Governance products.	Guardrails are not described on this page. Which governance features are in the price, and which are separate products?
Pydantic Logfire	Observability and evaluation on OpenTelemetry. One view for LLM calls, tool calls, database queries and API requests. Cost and latency per model and agent. Datasets from production traces, with code, LLM-judge or human review. Alerts on any queryable metric, such as an eval score drop or a spend jump.	SQL-queryable data and an MCP server for coding agents to query it. Prompts and model settings changeable without a redeploy. SDK is MIT-licensed. Self-hosting is Enterprise plan only. Python, TypeScript and Rust SDKs.	Self-hosted only on the Enterprise plan: is that required for CSOB data residency? What is retention on lower tiers?

Tool	What the page says it does	Notable points	Ask the vendor
Confident AI (and DeepEval)	Evaluation, tracing and production monitoring. Failures are grouped into issues with alerts on quality or latency regressions. DeepEval runs regression tests on each pull request and fails the build below thresholds. Red teaming through the open-source DeepTeam.	Closest of the four to "implement and enforce" because of CI quality gates and red teaming. Git-based prompt versioning and governance policies. Cloud or self-hosted, with SOC 2 Type II, HIPAA and GDPR stated.	Runtime blocking is not described on the page. What does the governance feature do, and who maintains DeepTeam and DeepEval?

Sources: datadoghq.com/dg/ai/agent-observability, orq.ai/platform/observability-monitoring, pydantic.dev/logfire, confident-ai.com. Read on 5 November 2026. "Not described" means the page did not say, not that the product lacks it.

What each page actually states

A quick comparison of the four against the capabilities the course cared about. A dash means the page did not mention it; check with the vendor before assuming either way.

Capability	Datadog	Orq.ai	Logfire	Confident AI
Agent and tool-call tracing	Yes	Yes	Yes	Yes
OpenTelemetry	Not stated	Yes	Yes (built on it)	Not stated
Cost and token monitoring	Yes	Yes	Yes	Yes (token cost per call)
Alerts on latency, errors, spend or quality	Yes	Yes	Yes	Yes
Evaluation of production traces	Yes	Linked to separate product	Yes	Yes
Offline experiments before rollout	Yes	Not stated	Yes	Yes
CI gate on pull requests	Not stated	Not stated	Not stated	Yes (DeepEval)
Human review route	Human annotation	Yes, with audit trail	Human review option	Annotation by experts
Red teaming	Not stated	Not stated	Not stated	Yes (DeepTeam)
Runtime guardrails that block	Separate "AI Guard" product, no detail	Not described	Not stated	Not described
PII masking and logging controls	Access controls mentioned	Yes	Not stated	Not stated
Self-hosting	Not stated	Not stated	Enterprise plan	Yes, including VPC and on-prem
Prompt versioning or management	Not stated	Not stated	Yes	Yes (git-based)

The pattern to notice: strong on seeing and measuring, thin on stopping. That matches the Day 4 point that guardrails need their own tuning, test set and owner.

Other tools, by job

Observe: tracing, cost and alerts

Tool	Type	Worth knowing
Langfuse	Open source tracing, evals, prompt management	MIT core and self-hostable. Described in the Day 4 handout as owned by ClickHouse since January 2026. Self-hosting needs Postgres and ClickHouse.
LangSmith	LangChain, proprietary	Deepest fit with LangChain and LangGraph. Per-seat pricing; check the self-host option for residency.
Arize Phoenix	OpenTelemetry-native tracing and evals	Runs as a single container. Sources disagree on its licence (Elastic License 2.0 in most, Apache 2.0 in one), so check the repository. Arize is part of Dynatrace per the Day 4 handout.
Helicone	Proxy-based gateway with observability	Quick to add because it sits in front of model calls. Adds a hop in the call path, so its availability matters.

Tool	Type	Worth knowing
Braintrust	Evaluation and observability platform	Often used to keep red-team findings and production cases in one evaluation record.
OpenTelemetry GenAI conventions	Standard for trace attributes	Still marked Development. Choosing tools that speak OpenTelemetry limits lock-in.

Enforce: guardrails, gateways and tool access

Tool	Type	Worth knowing
Amazon Bedrock Guardrails	Managed service	Can be applied to models outside Bedrock through its ApplyGuardrail API.
Azure AI Content Safety, Prompt Shields	Managed service	Jailbreak and indirect prompt injection detection, plus groundedness detection. Prompt Shields now sits within Microsoft Foundry guardrails.
NVIDIA NeMo Guardrails	Open source (Apache 2.0)	Programmable conversation rails and topic control. It controls what the agent says, not whether it is true.
Guardrails AI	Open source validators	Composable checks on model output. The company was bought by Harvey in September 2026: ask what that means for the open-source version.
Check Point AI Guardrails (formerly Lakera Guard)	Commercial API	Prompt injection and data leakage detection with low latency.
Llama Guard 4	Open model	Check its licence terms, which include an EU clause.
LLM Guard	Archived	Archived by Protect AI on 9 July 2026. Do not start new work on it.
LLM gateways (Portkey, Helicone, Bifrost)	Central enforcement point	One place for spend limits, filtering and logging, rather than each application doing its own.
MCP gateways (Traefik Hub, MintMCP, WitnessAI, TrueFoundry)	Control between agents and tools	Authenticate the agent, allow specific tools per role, inspect calls and keep an audit trail. This is where Day 2 tool approval and scopes become enforceable.

Prove and govern: red teaming, testing and inventory

Tool	Type	Worth knowing
garak (NVIDIA)	Open source vulnerability scanner	Broad probe library for injection, jailbreaks and leakage. Scans only; no guardrails.
PyRIT (Microsoft)	Open source, MIT	Multi-turn and adaptive attack campaigns, good for custom scenarios.
promptfoo	Open source CLI, MIT core	Evaluation and red teaming from YAML config, easy to run in CI. OpenAI announced its acquisition in March 2026; some teams weigh vendor independence.
DeepTeam, DeepEval	Open source (Confident AI)	Attack runs mapped to the OWASP Top 10, alongside regression tests.
Giskard, Inspect (UK AI Security Institute)	Test suites	Giskard positions itself on EU sovereignty; Inspect comes from a public body.
AI security and governance platforms	Commercial	Inventory, shadow AI discovery and policy evidence. Heavy consolidation: Palo Alto Networks bought Protect AI, F5 bought CalypsoAI, Check Point bought Lakera. Ask who owns the product now.

These give breadth and repeatability, not proof of safety. Ask the six vendor questions from Day 4 about scope, version, language, who judged, re-run triggers and data handling.

Mapping the twelve controls to tools

This is the control ownership matrix from Day 4, with the kinds of tool that help each one. The last column is the question that matters most: the tool can exist and the control can still be missing.

Control (day)	Tools that help	What still needs a person
Retrieval access trimming (1)	Permission filtering in the search or vector store; retrieval spans in a tracing tool to check it	Someone who owns the document permissions and tests that a user cannot retrieve what they could not open
Tool and MCP server approval and scopes (2)	MCP gateways with a registry of approved servers and per-tool policy	A named approver and a review date for each server
Boundary policy decided outside the agent (2)	Gateway or platform policy, cloud permissions, the Rule of Two as a design check	Agreeing the policy before the build, not in the prompt
Gateway filtering of prompts and responses (3)	LLM gateway plus a guardrail service (Bedrock Guardrails, Prompt Shields, NeMo, Check Point)	Who tunes it and who notices when it stops filtering
System prompt change control (3)	Git, prompt versioning in Logfire or Confident AI, trace tags with the prompt revision	A change owner and a re-test before release
Escalation review (3)	Human review queues (Orq.ai), alerts into an on-call route, annotation in Datadog or Confident AI	The reviewer. This was the gap with no owner on Day 3
Configuration record and re-testing (3)	DeepEval or promptfoo in CI, Datadog experiments, pinned model versions	Keeping the labelled test set current
Tracing and alerting (4)	Datadog, Orq.ai, Logfire, Confident AI, Langfuse, Phoenix, LangSmith	The first check on call, written down, and a tested alert
Cost monitoring and caps (4)	Gateway budgets, cloud billing alerts, cost alerts in the observability tools, iteration caps in code	Who can stop it at 03:00
Residency and subprocessor checks (4)	Self-hosting (Langfuse, Phoenix, Logfire Enterprise, Confident AI), PII masking, region settings	Whether the trace store holds a copy of customer data, and for how long
Guardrail tuning and monitoring (4)	A labelled test set run against the guardrail, red-team tools, evaluation platforms	What it misses and wrongly blocks, reviewed on a schedule
Approved tools and staff data-entry rules (4)	Shadow AI discovery and data loss prevention in security platforms	Policy, training and a line staff understand

A sensible starting stack

No single product covers everything. For a bank with data residency and audit demands, a layered setup is easier to defend than one platform:

Layer	Starting point	Why
Traces	OpenTelemetry into whatever the organisation already runs, with one agent-aware tool on top (self-hosted if residency demands it)	Keeps the data in your boundary and limits lock-in
Gates before release	DeepEval or promptfoo in CI against a labelled test set, run after any prompt, model or tool change	Makes "re-test after change" automatic
Runtime	An LLM gateway and an MCP gateway as the enforcement points, with a guardrail service behind the gateway	Enforcement in one place, with one audit trail
Periodic proof	garak or PyRIT campaigns, plus a manual round, scheduled with a named owner	Automated runs miss what a person finds

Before buying anything, take the twelve-control matrix back to the room and fill the "who runs it" column. The tool choice follows from the owners, not the other way round.

Sources: vendor pages as listed under the four tools; Day 4 "where this lives" handout (owners and statuses checked 4 November 2026); comparison articles on LLM observability, guardrails, red teaming and MCP gateways from 2026 (including MLflow, SigNoz, Braintrust, appsecsanta, Traefik, API7 and WitnessAI), several of which are written by vendors.